AULA 05
ESTRUTURAS DE DADOS

Programação em Python • Parte 1 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Estruturas de Dados • 8 horas
Professor: Msc. Hygo Sousa de Oliveira

(95) 99173-5676 / (92) 99171-0391
www.iteam.org.br
Av. Glaycon de Paiva nº 1820,
Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00

Roteiro da aula

O que vamos ver nesta aula

1. Visão geral das coleções nativas
2. Listas: criação, fatiamento e métodos
3. Tuplas: imutabilidade e uso
4. Dicionários: acesso, métodos e aninhamento
5. Conjuntos: duplicatas e operações
6. Exemplos guiados e preparação do checkpoint

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Escolher a coleção certa para cada problema**
  Lista, tupla, dicionário ou conjunto
- **Criar e manipular listas**
  Índices, fatias e métodos
- **Usar tuplas, dicionários e conjuntos**
  Imutabilidade, chave-valor e operações de conjuntos
- **Prever o comportamento de um código**
  Preparação para o checkpoint do Módulo 3

Módulo 3 – Estruturas de Dados

- Listas: criação, manipulação, métodos
- Tuplas: criação, imutabilidade, uso
- Dicionários: criação, acesso, métodos
- Conjuntos: criação, operações, uso
- Compreensões e funções lambda (Aula 6)

Quatro coleções nativas

Cada estrutura resolve um tipo de problema

Estrutura	Sintaxe	Ordem	Mutável	Repetidos
`list`	`[1, 2, 3]`	Sim	Sim	Permite
`tuple`	`(1, 2, 3)`	Sim	Não	Permite
`dict`	`{"a": 1}`	Por inserção	Sim	Chaves únicas
`set`	`{1, 2, 3}`	Não	Sim	Não permite

DICA Pergunte: preciso de ordem? vou alterar? preciso buscar por chave? os itens são únicos?

Listas: criação e acesso

Ordenadas, mutáveis e capazes de guardar qualquer tipo

listas.py

```
alunos = ["Ana", "Bruno", "Carlos"]
dados_mistos = [1, "Teste", True, 3.14]
print(alunos[0], alunos[-1])
print(len(alunos), "Bruno" in alunos)
print(dados_mistos)
```

SAÍDA

```
Ana Carlos
3 True
[1, 'Teste', True, 3.14]
```

DICA Índices começam em 0. Índices negativos contam do fim: `-1` é o último elemento.

Listas: fatiamento

lista[inicio:fim:passo], com o fim exclusivo

fatias.py

```
numeros = [10, 20, 30, 40, 50]
print(numeros[1:4])
print(numeros[:2])
print(numeros[::2])
print(numeros[::-1])
```

SAÍDA

```
[20, 30, 40]
[10, 20]
[10, 30, 50]
[50, 40, 30, 20, 10]
```

NOTA Fatiar cria uma **nova** lista e nunca gera erro de índice, mesmo que o intervalo passe do fim.

Listas: métodos de manipulação

Adicionar, inserir, remover e ordenar

metodos.py

```
alunos = ["Ana", "Bruno", "Carlos"]
alunos.append("Daniela")
alunos.insert(1, "Bárbara")
alunos.remove("Carlos")
ultimo = alunos.pop()
alunos.sort()
print(alunos, ultimo)
```

SAÍDA

```
['Ana', 'Bruno', 'Bárbara'] Daniela
```

ATENÇÃO `sort()` compara pelo código do caractere: `Bárbara` vem depois de `Bruno`.

Métodos de lista: resumo

Os mais usados no dia a dia

Método	O que faz
`append(x)`	Adiciona x no fim da lista
`insert(i, x)`	Insere x na posição i
`remove(x)`	Remove a primeira ocorrência de x
`pop(i)`	Remove e devolve o item da posição i (padrão: o último)
`sort()`	Ordena a própria lista
`reverse()`	Inverte a ordem da lista
`count(x)`	Conta quantas vezes x aparece
`index(x)`	Posição da primeira ocorrência de x

Listas: cópia e ordenação

Cuidado: atribuir uma lista não a copia

copia.py

```
notas = [7.5, 9.0, 6.0]
print(sorted(notas), notas)
copia = notas.copy()
alias = notas
alias.append(10.0)
print(notas)
print(copia)
```

SAÍDA

```
[6.0, 7.5, 9.0] [7.5, 9.0, 6.0]
[7.5, 9.0, 6.0, 10.0]
[7.5, 9.0, 6.0]
```

ATENÇÃO `b = a` faz as duas variáveis apontarem para a **mesma** lista. Para copiar, use `a.copy()` ou `a[:]`. `sorted()` devolve nova lista; `sort()` altera a original.

Tuplas: criação e imutabilidade

Sequências que não podem ser alteradas depois de criadas

tupla.py

```
coordenadas = (-23.5505, -46.6333)
lat, lon = coordenadas
print(lat, lon)
coordenadas[0] = -24.0
```

SAÍDA

```
-23.5505 -46.6333
Traceback (most recent call last):
  File "tupla.py", line 4, in <module>
    coordenadas[0] = -24.0
    ~~~~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

NOTA Imutável: mais segura contra alterações acidentais e levemente mais rápida que a lista.

Tuplas: quando usar

Dados fixos, desempacotamento e chaves de dicionário

tupla_uso.py

```
ponto = (3, 4)
x, y = ponto
a, b = 1, 2
a, b = b, a
mapa = {(0, 0): "origem", (3, 4): "ponto A"}
print(mapa[ponto], a, b)
print(len(ponto), ponto.count(3), (5,))
```

SAÍDA

```
ponto A 2 1
2 1 (5,)
```

DICA Tupla de um elemento exige vírgula: `(5,)`. Tuplas podem ser chaves de dicionário; listas não.

Dicionários: criação e acesso

Pares chave-valor com busca rápida

dicionario.py

```
usuario = {
    "id": 101,
    "nome": "Marcos Silva",
    "email": "marcos@iteam.org",
    "ativo": True,
}
print(usuario["nome"])
print(usuario.get("telefone", "Não informado"))
usuario["idade"] = 28
print(len(usuario))
```

SAÍDA

```
Marcos Silva
Não informado
5
```

ATENÇÃO `usuario["x"]` gera KeyError se a chave não existir; `get()` devolve um valor padrão.

Dicionários: métodos e iteração

keys(), values(), items(), update() e pop()

dict_metodos.py

```
usuario = {"nome": "Marcos", "idade": 28}
print(usuario.keys())
print(usuario.values())
for chave, valor in usuario.items():
    print(f"{chave} = {valor}")
usuario.update({"cidade": "Boa Vista"})
usuario.pop("idade")
print(usuario)
```

SAÍDA

```
dict_keys(['nome', 'idade'])
dict_values(['Marcos', 28])
nome = Marcos
idade = 28
{'nome': 'Marcos', 'cidade': 'Boa Vista'}
```

Lista de dicionários

A estrutura mais comum para representar registros

registros.py

```
alunos = [
    {"nome": "Ana", "nota": 9.0},
    {"nome": "Bruno", "nota": 6.5},
]
total = 0
for aluno in alunos:
    print(aluno["nome"], aluno["nota"])
    total += aluno["nota"]
print(f"Média: {total / len(alunos):.2f}")
```

SAÍDA

```
Ana 9.0
Bruno 6.5
Média: 7.75
```

NOTA É a mesma forma de um arquivo JSON, que vamos usar mais adiante para guardar dados em arquivos.

Conjuntos: criação e duplicatas

Coleções sem ordem e sem itens repetidos

conjuntos.py

```
numeros = [1, 2, 2, 3, 3, 4, 5, 5]
conjunto = set(numeros)
print(conjunto, len(conjunto))
conjunto.add(6)
conjunto.discard(1)
print(sorted(conjunto), 3 in conjunto)
```

SAÍDA

```
{1, 2, 3, 4, 5} 5
[2, 3, 4, 5, 6] True
```

ATENÇÃO `{}` cria um dicionário vazio; para um conjunto vazio use `set()`.

Conjuntos: operações

União, interseção, diferença e diferença simétrica

operacoes.py

```
turma_a = {"Ana", "Bruno", "Carlos"}
turma_b = {"Carlos", "Daniela", "Eduardo"}
print(sorted(turma_a | turma_b))
print(turma_a & turma_b)
print(sorted(turma_a - turma_b))
print(sorted(turma_a ^ turma_b))
```

SAÍDA

```
['Ana', 'Bruno', 'Carlos', 'Daniela', 'Eduardo']
{'Carlos'}
['Ana', 'Bruno']
['Ana', 'Bruno', 'Daniela', 'Eduardo']
```

NOTA Operadores: `|` união, `&` interseção, `-` diferença, `^` diferença simétrica (métodos: `union`, `intersection`, `difference`).

Exemplo guiado 1: notas da turma

Lista + laço + funções embutidas

DESAFIO Leia três notas e mostre a maior, a menor, a média e as notas em ordem decrescente.

notas_lista.py

```
notas = []
for i in range(1, 4):
    notas.append(float(input(f"Nota {i}: ")))
print(max(notas), min(notas))
print(f"Média: {sum(notas) / len(notas):.2f}")
print(sorted(notas, reverse=True))
```

SAÍDA

```
Nota 1: 7.5
Nota 2: 9
Nota 3: 6
9.0 6.0
Média: 7.50
[9.0, 7.5, 6.0]
```

Exemplo guiado 2: agenda de contatos

Dicionário como banco de dados em memória

DESAFIO Cadastre dois contatos (nome e telefone) e depois busque um deles pelo nome.

agenda.py

```
agenda = {}
for _ in range(2):
    nome = input("Nome: ")
    agenda[nome] = input("Tel: ")
busca = input("Buscar: ")
print(agenda.get(busca, "Não encontrado"))
print(f"Total: {len(agenda)}")
```

SAÍDA

```
Nome: Ana
Tel: 9999-1111
Nome: Bruno
Tel: 9888-2222
Buscar: Bruno
9888-2222
Total: 2
```

Exemplo guiado 3: convidados com conjuntos

Operações de conjuntos em um caso real

DESAFIO Duas listas de presença (manhã e tarde): quem veio nos dois turnos, quem veio só de manhã e o total de pessoas.

presenca.py

```
manha = {"Ana", "Bruno", "Carla"}
tarde = {"Bruno", "Diego", "Carla"}
print("Nos dois turnos:", sorted(manha & tarde))
print("Só de manhã:", sorted(manha - tarde))
print("Total de pessoas:", len(manha | tarde))
```

SAÍDA

```
Nos dois turnos: ['Bruno', 'Carla']
Só de manhã: ['Ana']
Total de pessoas: 4
```

Atividades da Aula 5

Módulo 3: prática guiada e preparação do checkpoint

- **Prática guiada em sala**
  Refaça os três exemplos trocando os dados (notas, contatos e convidados)
- **Preparação para o checkpoint**
  Revise listas, tuplas, dicionários, conjuntos e leitura de código
- **Checkpoint de Estruturas de Dados (10 questões)**
  Múltipla escolha, aplicado ao final do Módulo 3 (Aula 6) e entregue via GitHub

GIT Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Próxima aula: Compreensões, lambda e o padrão Registry
ITEAM – Instituto Tecnológico Educacional da Amazônia